AI Backend Development Proposal

Question Generator & Rulebook Intelligence

For StuddyBuddyPro (Final Combined Version)

---

1. Project Overview

StuddyBuddyPro aims to introduce an AI-powered question generation system for professors. This system will allow professors to generate structured academic questions for student practice based on predefined legal/academic frameworks.

Professors will provide contextual inputs such as:

- Question Title
- Question Type (IRAC / IREAC / Custom types)
- Course Selection
- Topic Selection
- Rulebook references
- Story / scenario context

The backend AI system will then:

- Assist in rulebook reference discovery from previously stored question data.
- Generate structured practice questions including scenario stories, questions, and model answers.

Additionally, the system will intelligently handle scenarios where data is limited or unavailable using a multi-layer AI fallback mechanism.

The work under this proposal is limited strictly to backend AI functionality and integration.

---

2. Scope of Work

2.1 AI Rulebook Reference Engine (RAG System)

When a professor selects a course and topic, the system will:

- Fetch previously stored question data from WordPress database tables.
- Extract rulebook references and phrases previously used in similar questions.
- Store processed content in a vector database.
- Use RAG (Retrieval Augmented Generation) to suggest:
  - Relevant rulebook phrases
  - Keywords
  - Legal reasoning references
  - Contextual supporting material

Edge Case Handling (Critical Enhancement)

The system will handle three scenarios:

Case 1: Question Dataset Available (Primary Flow)

- Use RAG on historical questions
- Extract rulebook phrases directly
- Confidence: **High**

Case 2: No Questions, But Case Briefs Available

- Retrieve case briefs using semantic search
- Extract:
  - Legal rules
  - Reasoning patterns
  - Terminology
- Convert into structured rulebook
- Confidence: **Medium**

Case 3: No Questions and No Case Briefs

- Use LLM with controlled prompts
- Generate only standard legal doctrines
- Apply strict validation
- Confidence: **Low**
- System may return: *"Insufficient Data"* or request professor input

---

2.2 AI Question Generator

After professor inputs are finalized, the system will generate:

- Scenario / Story
- Questions students must answer

- Model answer following selected format (IRAC / IREAC / custom)

Custom Format Handling (Important Capability)

The system will support dynamic professor-defined formats:

- Professors can define custom answer structures
- Example:
    - Issue → Analysis → Conclusion
    - Policy-based reasoning

System behavior:

- Convert custom format into structured template
- Inject into LLM prompt dynamically
- Ensure generated answers follow defined structure

Technologies

- Claude model for generation
- GPT models for retrieval reasoning
- ChromaDB (Vector Database)
- Django backend for orchestration

2.3 Backend APIs

Backend APIs will be developed to support:

- Course and topic input handling
- Rulebook reference retrieval
- AI question generation
- Response formatting for frontend

Swagger API documentation will be implemented.

2.4 Scoring Engine for New Questions

- Replicate existing scoring API
- Modify scoring mechanism per new structure
- Handle section-wise scoring

- Provide scoring APIs

---

3. Out of Scope

- Frontend UI development
- WordPress schema modifications
- Manual legal validation

---

4. Technical Approach

Step 1: Data Extraction

- Extract question data from WordPress

Step 2: Data Processing

- Clean, chunk, and embed data

Step 3: Vector Storage

- Store embeddings in vector DB

Step 4: Retrieval System

- Retrieve from:
  - Question dataset
  - Case briefs
  - Fallback to LLM if needed

Step 5: Prompt Construction

- Combine professor input + rulebook

Step 6: AI Generation

- Generate story, question, and answers

Step 7: Scoring Engine

- Evaluate answers using scoring API

---

5. Guardrails & Validation

Key Safeguards

- Confidence-based output filtering
- Minimum rulebook requirement (≥3 phrases)
- Structured output validation
- Second-pass LLM validation
- Hallucination prevention checks

---

6. Testing Strategy

Functional Testing

- API validation

AI Output Testing

- Structure correctness
- Format validation

Guardrail Testing

- No hallucinated outputs

Edge Case Testing

- Missing data scenarios
- Custom format handling

---

7. Assumptions

- WordPress access available
- Structured data exists
- Hosting provided

---

8. Sprint Plan

Sprint 1 — System Setup

- WordPress data access
- Data extraction pipeline
- Data cleaning and preprocessing
- Embedding generation pipeline

Sprint 2 — RAG Implementation

- Vector database setup
- Semantic retrieval implementation
- Rulebook suggestion API
- Edge case implementation

Sprint 3 — Question Generation Engine

- Prompt structure implementation
- Claude integration
- Generation pipeline

Sprint 4 — Integration & Guardrails

- Backend API integration
- Output validation
- Guardrails implementation

Sprint 5 — Scoring Mechanism API

- Replicate the existing scoring mechanism api
- Update the scoring mechanism as per new scoring method
- Handle the background process for scoring process

Sprint 6 — Testing & Refinement

- Functional testing
- AI output testing

- Edge case handling
- System stabilization

10. Milestone Deliverables

Milestone	Description	Key Deliverables	Timeline	Cost (USD)
Milestone 1: RAG for Rulebook Phrases	Implementation of Retrieval Augmented Generation system to fetch and suggest relevant rulebook phrases and keywords based on selected course and topic.	- WordPress data extraction- Embedding generation pipeline- Vector database setup- Rulebook phrase retrieval API	5 Days	
Milestone 2: Question Generation Pipeline Implementation	Development of AI pipeline to generate structured questions, scenarios, and model answers using professor inputs and rulebook references.	- Prompt orchestration system- Claude integration for question generation- Story + question + answer generation logic- API for question generation	8 Days	
Milestone 3: Scoring Mechanism API	Replicating Existing scoring API and updating the new scoring mechanism	- Scoring and validation based on new score method and algos	4 Days	
Milestone 3: UAT and Deployment	System testing, validation, bug fixes, and deployment support.	- Functional testing- AI output testing- Guardrails validation- UAT support with client team- Deployment assistance	8 Days	

---

10. Total Cost

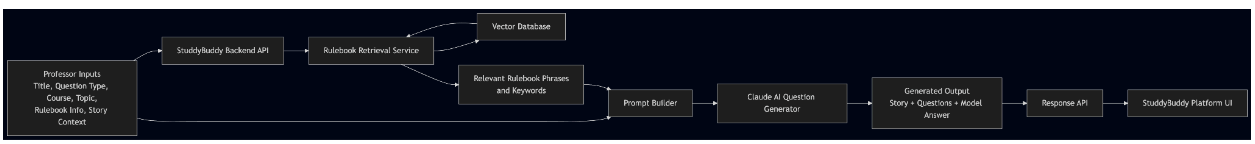
11. Key Value Addition

- Handles missing data scenarios intelligently
- Supports custom teaching formats
- Reduces hallucination risks
- Scalable for new courses/topics

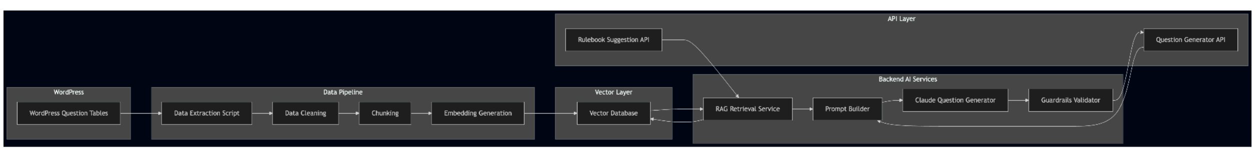
End of Final Combined Proposal

12. Flow Diagrams

High Level Flow



Detailed Architecture



Edge Cases Handling Flow